UML Class Diagrams — Complete Notes for Low-Level Design

1. Why Do We Need UML Diagrams?

In a small application, we can often write classes directly and understand how they fit together. As the system grows, keeping every class, responsibility, and relationship in our head becomes difficult.

Consider designing a **Library Management System**. It may contain classes such as `Member`, `Book`, `BookCopy`, `Loan`, and `LibraryBranch`. Understanding each class individually is not enough; we also need to understand how they connect.

A diagram helps us:

- **Visualize the design:** See classes and relationships without reading the entire codebase.
- **Communicate the design:** Explain the structure to teammates or interviewers.
- **Spot problems early:** Identify missing relationships, unclear responsibilities, or unnecessary coupling before implementation.

A diagram does not replace code. It gives us a simpler view of the decisions that matter.

What is UML?

UML (Unified Modeling Language) is a standardized visual language for describing software systems. It is closely associated with object-oriented design, but it can also describe interactions, components, and deployments.

For low-level design (LLD), three useful diagram types commonly studied together are:

Diagram	What it explains
Class diagram	The static structure: classes, attributes, operations, and relationships
Sequence diagram	How participants interact over time
Entity Relationship (ER) diagram	How data entities are organized and related in a database

Note: ER diagrams are not technically UML diagrams; they are commonly taught alongside UML for software design.

UML is a way to **express** a design, not a guarantee that the design is good. A valid diagram can still describe a tightly coupled or poorly organized application.

---

2. Anatomy of a UML Class

A class is usually drawn as a rectangle with three compartments:

```
+----------------------------------+
| BookCopy                         |  <- Class name
+----------------------------------+
| - id: long                       |
| - book: Book                     |  <- Attributes
| - available: boolean             |
+----------------------------------+
| + borrow(): void                 |
| + returnBook(): void             |  <- Operations
+----------------------------------+
```

2.1 Class name

The top compartment identifies the class. Class names are generally singular and follow the naming conventions of the programming language, such as `BookCopy` or `LibraryBranch`.

2.2 Attributes

The middle compartment lists the data associated with an object. UML writes the **attribute name first, followed by its type**:

```
- id: long
- book: Book
- available: boolean
```

2.3 Operations

The bottom compartment lists the behavior exposed by the class. UML writes the **return type after the parameter list**:

```
+ borrow(): void
+ returnBook(): void
```

Equivalent Java code:

```
public class BookCopy {
    private long id;
    private Book book;
    private boolean available;

    public void borrow() {
        available = false;
    }

    public void returnBook() {
        available = true;
    }
}
```

Java	UML
`private long id;`	`- id: long`
`private boolean available;`	`- available: boolean`
`public void borrow()`	`+ borrow(): void`
`public boolean isAvailable()`	`+ isAvailable(): boolean`

A class diagram does **not** need to contain every field or method. Include the details that help explain the design; omit implementation details that would make the diagram harder to read.

---

3. Important UML Class Notations

3.1 Visibility / access modifiers

Symbol	Java equivalent	Meaning
`+`	`public`	Accessible from other classes, subject to Java access rules
`-`	`private`	Accessible within the declaring class
`#`	`protected`	Accessible under Java's protected-access rules
`~`	package-private	Accessible within the same package in Java

These symbols are useful when a diagram needs to communicate **encapsulation**. In an interview, not every member needs a visibility symbol if the extra detail does not help explain the design.

3.2 Method parameters and return types

Java:

```
public Loan borrowBook(long memberId, long copyId) {
    // Implementation
}
```

UML:

```
+ borrowBook(memberId: long, copyId: long): Loan
```

For a method with no return value:

```
+ returnBook(loanId: long): void
```

The general form is:

```
visibility methodName(parameterName: Type): ReturnType
```

3.3 Constructors

A constructor may be shown as an operation with the class name:

```
public class BookCopy {
    private final long id;
    private final Book book;

    public BookCopy(long id, Book book) {
        this.id = id;
        this.book = book;
    }
}
```

UML:

```
+ BookCopy(id: long, book: Book)
```

Constructors are worth showing when object creation or required dependencies are important to the design. Otherwise, they can be omitted.

3.4 Static attributes and methods

A **static** field or method belongs to the class rather than an individual instance.

```
public class LibraryConfig {
    private static final int MAX_LOANS = 5;

    public static int getMaxLoans() {
        return MAX_LOANS;
    }
}
```

Standard UML notation **underlines static attributes and operations**. For example, `getMaxLoans(): int` would be underlined in the diagram. Some tools use annotations instead.

3.5 Abstract classes

An abstract class provides shared structure or behavior while leaving some operations for subclasses to implement.

```
public abstract class LibraryItem {
    private long id;
    private String title;

    public abstract int getMaxBorrowDays();
}
```

```
public class Book extends LibraryItem {
    @Override
    public int getMaxBorrowDays() {
        return 14;
    }
}

public class Magazine extends LibraryItem {
    @Override
    public int getMaxBorrowDays() {
        return 7;
    }
}
```

In UML, abstract class and operation names may be written in *italics*; the class can also be labeled `<<abstract>>` for clarity.

`Book` and `Magazine` are specialized versions of `LibraryItem`. The **solid line with a hollow triangle** points toward the parent class.

3.6 Interfaces

An interface describes a contract that implementing classes must follow. In UML, interfaces can be marked using the stereotype `<<interface>>`.

```
public interface NotificationSender {
    void send(String message);
}
```

When a class implements an interface, UML uses a **dashed line with a hollow triangle pointing toward the interface**. This is called **realization**.

---

4. Relationships Between Classes

Objects rarely work in isolation. A member borrows a book copy; a branch stores copies; an order contains line items. These connections are represented through class relationships.

Relationship	Meaning	Common notation
Association	Objects have a structural connection	Solid line
Aggregation	Shared or weak whole–part relationship	Solid line, hollow diamond at the whole
Composition	Strong whole–part relationship with exclusive ownership	Solid line, filled diamond at the whole
Generalization	An *is-a* / inheritance relationship	Solid line, hollow triangle toward parent
Realization	A class fulfills an interface contract	Dashed line, hollow triangle toward interface
Dependency	One element uses or relies on another	Dashed arrow toward the dependency

4.1 Association — a structural relationship

Example: A student enrolls in a course. The student is not a type of course and does not own the course, but they are related through enrollment.

In a library, a `Loan` connects a member to a physical book copy:

```
public class Loan {
    private Member member;
    private BookCopy bookCopy;
}
```

This creates structural associations between `Loan` and `Member`, and between `Loan` and `BookCopy`.

An association does **not** automatically imply ownership. A `Member` and a `BookCopy` exist independently of an individual loan.

4.2 Aggregation — a weak whole–part relationship

Example: A library branch has a collection of book copies.

```
public class LibraryBranch {
    private List<BookCopy> copies;
}
```

A book copy can still exist if the branch closes; it may simply be transferred to another branch. That makes a weak whole–part interpretation possible.

UML shows shared aggregation with a **hollow diamond next to** `LibraryBranch`, which is the whole.

Practical note: Shared aggregation has limited formal semantics in UML. Many designs use an ordinary association instead. Use aggregation only when the whole–part meaning communicates something useful; a field containing another object does not automatically require a hollow diamond.

4.3 Composition — strong ownership

Example: An order contains order line items.

```
Order #101
  |-- OrderItem: Laptop x 1
  |-- OrderItem: Mouse x 2
```

An `OrderItem` represents a line belonging to a specific `Order`. In this model, it does not have a separate business identity outside that order, and the same line item should not belong to two orders simultaneously.

UML shows composition with a **filled diamond next to** `Order`, the owning whole.

Composition communicates **exclusive composite ownership**. The lifetime of a part is conceptually tied to the composite, although the exact database deletion or object-memory behavior depends on the implementation.

4.4 Generalization — an “is-a” relationship

`Book` and `Magazine` are both kinds of `LibraryItem`.

```
Book       --inherits from--> LibraryItem
Magazine   --inherits from--> LibraryItem
```

In UML, each child connects to the parent with a **solid line ending in a hollow triangle at** `LibraryItem`.

This relationship communicates inheritance: the specialized class is a kind of the more general class.

4.5 Realization — implementing an interface

```
public interface NotificationSender {
    void send(String message);
}

public class EmailNotificationSender implements NotificationSender {
    @Override
    public void send(String message) {
        // Send email
    }
}
```

`EmailNotificationSender` fulfills the contract defined by `NotificationSender`.

The UML notation is a **dashed line with a hollow triangle pointing toward** `NotificationSender`.

4.6 Dependency — a "uses" relationship

A dependency exists when one element relies on another and may be affected by changes to it.

```
public class ReceiptService {
    public String generateReceipt(
            Loan loan,
            ReceiptFormatter formatter) {

        return formatter.format(loan);
    }
}
```

`ReceiptService` uses `ReceiptFormatter` to complete an operation but does not necessarily store a reference to a formatter as part of its state.

The notation is a **dashed arrow from** `ReceiptService` **to** `ReceiptFormatter`.

Association vs dependency

Association	Dependency
Describes a structural link between instances	Describes reliance on another element
Often represented by an object reference stored as a field	Often represented by a parameter or temporary use
Example: `LibraryService` stores a `LoanRepository` field	Example: `ReceiptService` accepts `ReceiptFormatter` as a method parameter

The distinction is about the relationship being modeled, not just whether a particular Java keyword appears.

---

5. Multiplicity and Cardinality

Relationships also tell us **how many objects** may participate at each end.

Suppose a `Book` represents a book title, while `BookCopy` represents a physical copy.

```
Book  "1" ---------------- "0..*"  BookCopy
```

This means:

- One `Book` may be associated with **zero or more** `BookCopy` objects.
- Each `BookCopy` is associated with **exactly one** `Book`.

Key rule: Read the multiplicity written near one class from the perspective of **one object at the other end**.

Common multiplicity symbols

Notation	Meaning	Example
`1`	Exactly one	Every loan has one member
`0..1`	Zero or one	A member may not yet have a membership card
`*`	Zero or more	A member may have many loans
`0..*`	Zero or more	Same meaning as `*`

Notation	Meaning	Example
`1..*`	One or more	An order must have at least one line item
`2..5`	Between two and five	A team contains two to five members

5.1 One-to-one

Suppose every member has one permanent membership card, and each card belongs to one member.

```
Member  "1" ---------------- "1"  MembershipCard
```

If the card can be issued later, the relationship may instead be:

```
Member  "1" ---------------- "0..1"  MembershipCard
```

A Java implementation could store references in one or both classes:

```
public class Member {
    private MembershipCard card;
}

public class MembershipCard {
    private Member member;
}
```

A **one-to-one multiplicity does not require bidirectional references**. Multiplicity and navigability describe different aspects of a relationship.

5.2 One-to-many

A member may have many borrowing records, but each record belongs to one member.

```
Member  "1" ---------------- "0..*"  Loan
```

There can be zero loans for a newly registered member, while each existing loan refers to exactly one member.

5.3 Many-to-many

A student may enroll in several courses, and a course may have several students.

```
Student  "0..*" ---------------- "0..*"  Course
```

The multiplicity at both ends allows multiple related objects. When designing a database, we often model this relationship through an `Enrollment` entity, but that is a separate database modeling decision.

Remember: Multiplicity follows the business rules. Neither one-to-one nor one-to-many relationships are automatically mandatory or optional; the `0` and `1` bounds tell us that.

---

6. Navigability: Who Can Access Whom?

Multiplicity answers **how many**. Navigability answers **which object can follow a relationship to reach another object**.

6.1 Unidirectional association

```
public class Loan {
    private Member member;
}
```

Here, `Loan` directly refers to `Member`. The `Member` class does not need a `List<Loan>` just because loans belong to members. We could retrieve a member's borrowing history through a repository:

```
loanRepository.findByMemberId(memberId);
```

This is a useful case for a **unidirectional association from `Loan` to `Member`**.

6.2 Bidirectional association

```
public class Member {
    private List<Loan> loans;
}

public class Loan {
    private Member member;
}
```

Here, both objects maintain references, so the relationship can be navigated in both directions.

The trade-off is **consistency**. If a loan is added to a member's list, the `Loan.member` reference must also identify the same member. Otherwise, the two views of the relationship disagree.

Bidirectional associations are useful when both directions are genuinely needed, but they should not be the default.

---

7. Practical Guidelines for LLD and Machine Coding Interviews

- **Start with the problem requirements.** Identify the important entities, their responsibilities, and how they interact.
- **Show useful details, not every implementation detail.** Class diagrams are most readable when they focus on meaningful design decisions.
- **Choose relationships deliberately.** Not every object reference represents aggregation or composition.
- **Distinguish structural associations from temporary dependencies.** Ask whether an object maintains a relationship or simply uses another type for an operation.
- **Check multiplicities from both ends.** For each relationship, ask how many objects may exist on either side and whether the relationship is optional.
- **Avoid unnecessary bidirectional references.** Add navigation only where the code actually needs it.

- **Use inheritance and interfaces where the model supports them.** Generalization represents an *is-a* relationship; realization represents fulfillment of a contract.
- **Treat UML as a communication tool.** A clean diagram helps explain a design, but good object-oriented decisions still matter more than diagram notation.

Quick recap

Concept	Main question it answers
Class	What objects/types are part of the design?
Attribute	What information does an instance hold?
Operation	What behavior does the class expose?
Association	Which objects are structurally connected?
Aggregation	Is there a meaningful shared whole–part relationship?
Composition	Does a whole exclusively own its parts?
Generalization	Is this class a specialized kind of another class?
Realization	Which interface contract does this class implement?
Dependency	Which other element does this class rely on?
Multiplicity	How many objects may be related at each end?
Navigability	In which direction does the model need to follow the relationship?

The purpose of a class diagram is to make the structure of a design understandable before getting lost in implementation details.